

Von Neumann Model, ISA, and LC-3

Computer Architecture

Last Lecture Recap

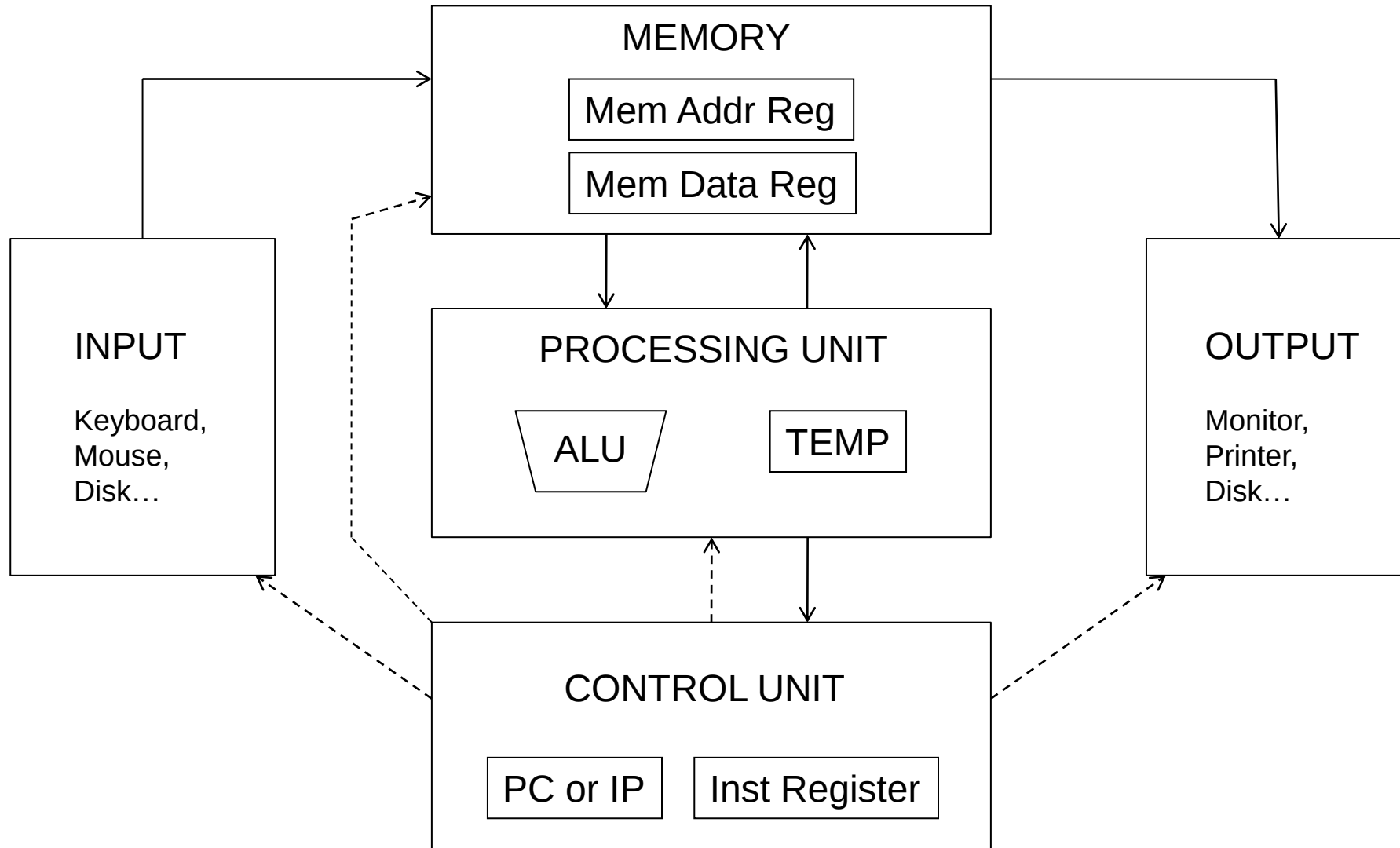
What Will We Learn?

- The von Neumann model
 - LC-3: An example von Neumann machine
- Instruction Set Architectures: LC-3
 - Operate instructions
 - Data movement instructions
 - Control instructions
- Instruction formats
- Addressing modes

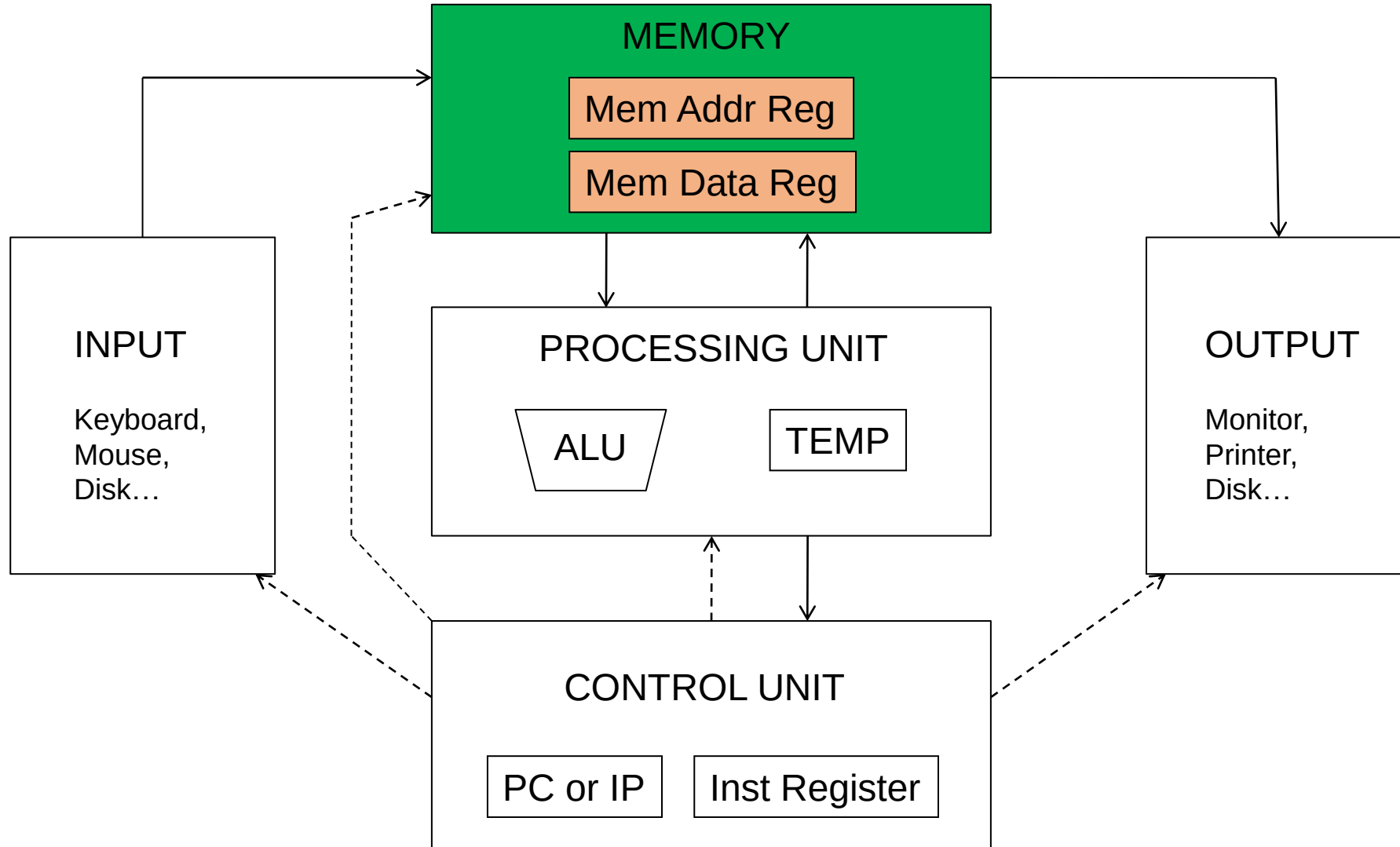
The Von Neumann Model

- John von Neumann proposed a fundamental model in 1946
- It consists of 5 parts
 - Memory
 - Processing unit
 - Input
 - Output
 - Control unit
- Throughout this lecture, we consider examples of the von Neumann model – LC-3

The Von Neumann Model



The Von Neumann Model



Memory

- The memory stores

- Data

- Programs

- The memory contains bits

- Bits are grouped into bytes (8 bits) and words (e.g., 8, 16, 32 bits)

- How the bits are accessed determines the addressability

- E.g., word-addressable

- E.g., 8-bit addressable (or byte-addressable)

- The total number of addresses is the address space

- In LC-3, the address space is 2^{16}

- 16-bit addresses

- In MIPS, the address space is 2^{32}

- 32-bit addresses

Readings

- *Introduction to Computing Systems: From Bits and Gates to C and Beyond* by Patt & Patel
 - Chapter 4, 5
 - Appendix A – LC3 ISA

Word-Addressable Memory

- Each **data word** has a **unique address**
 - In MIPS, a unique address for each **32-bit data word**
 - In LC-3, a unique address for each **16-bit data word**

Word Address	Data	MIPS memory
⋮	⋮	⋮
00000003	D 1 6 1 7 A 1 C	Word 3
00000002	1 3 C 8 1 7 5 5	Word 2
00000001	F 2 F 1 F 0 F 7	Word 1
00000000	8 9 A B C D E F	Word 0

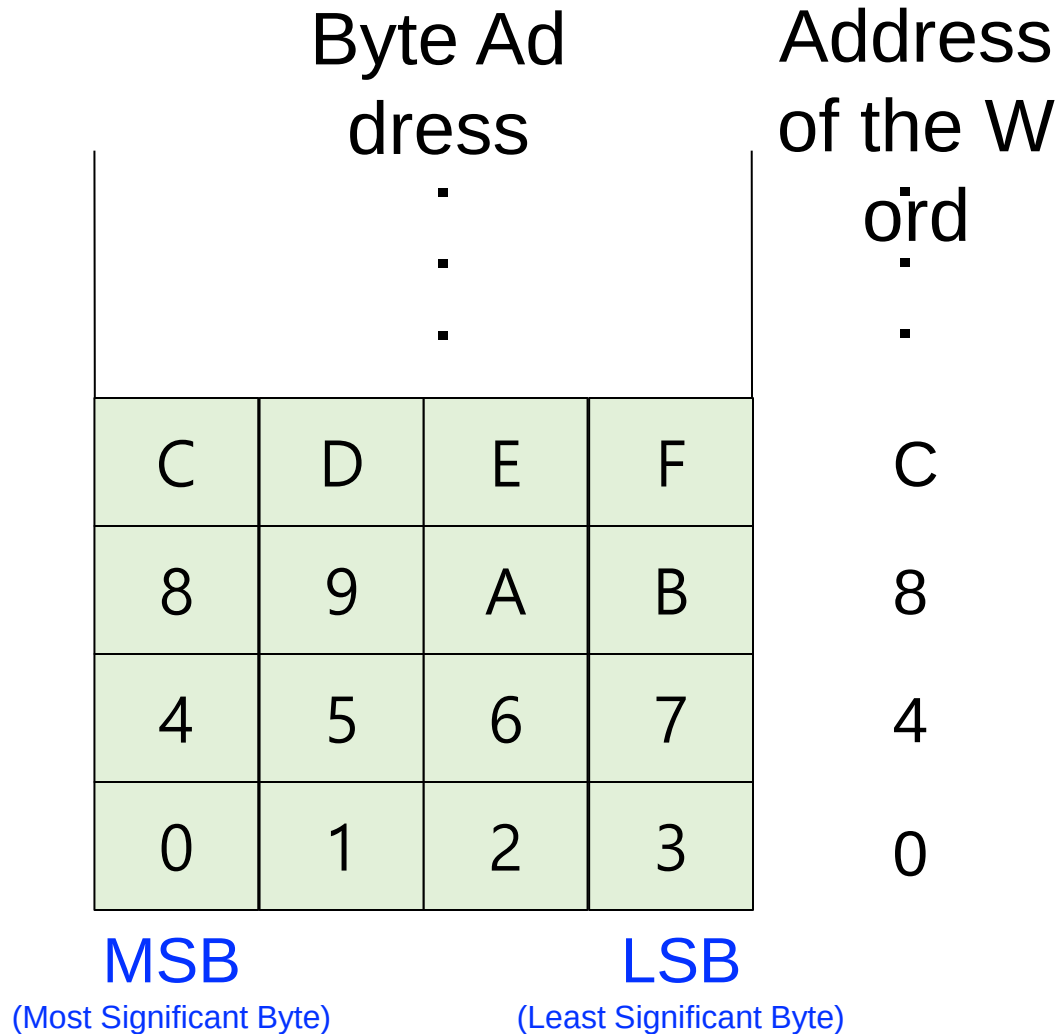
Byte-Addressable Memory

- Each **byte** has a **unique address**
 - Actually, MIPS is **byte-addressable**
 - LC-3b (updated version of LC-3) is **byte-addressable**, too

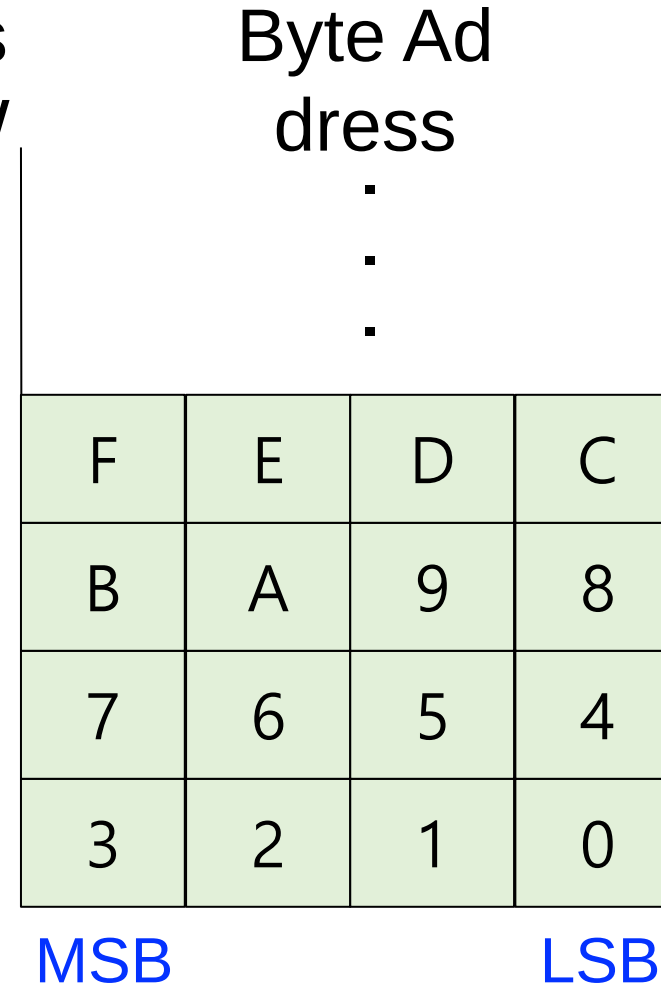
Byte Address of the Word	Data				MIPS memory
.	.	.	.	.	.
0000000C	D 1	6 1	7 A	1 C	Word 3
00000008	1 3	C 8	1 7	5 5	Word 2
00000004	F 2	F 1	F 0	F 7	Word 1
00000000	How are these four bytes addressed?				Word 0

Big Endian vs Little Endian

Big Endian



Little Endian



Big Endian vs Little Endian

Big Endian

Little Endian

Does this really matter?

Answer: No, it is a convention

Qualified answer: No, except when one big-endian system and one little-endian system have to share data

MSB

(Most Significant Byte)

LSB

(Least Significant Byte)

MSB

LSB

Accessing Memory: MAR and MDR

- There are two ways of accessing memory

- Reading or loading

- Writing or storing

- Two registers are necessary to access memory

- Memory Address Register (MAR)

- Memory Data Register (MDR)

- To read

- Step 1: Load the MAR with the address

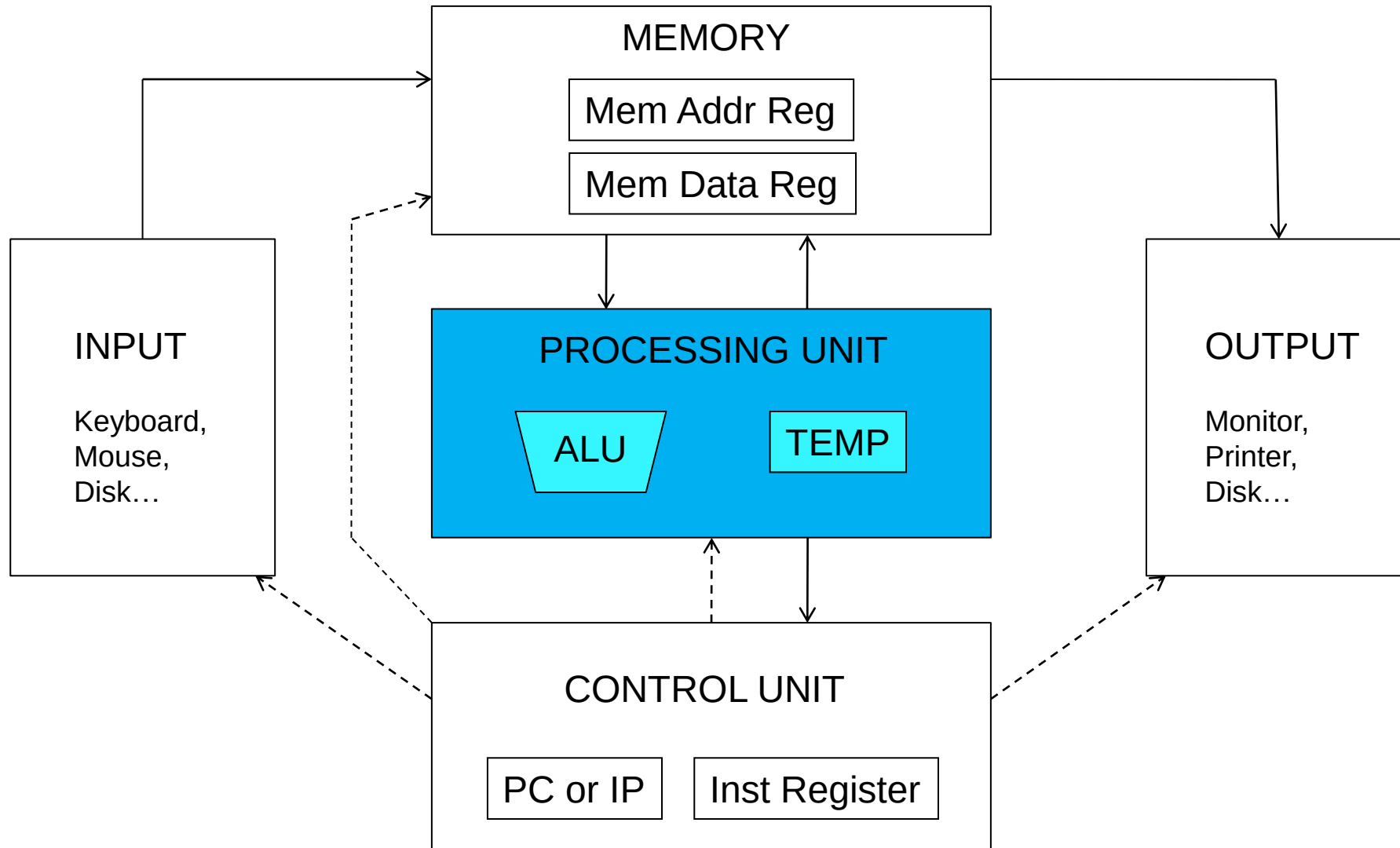
- Step 2: Data is placed in MDR

- To write

- Step 1: Load the MAR with the address and the MDR with the data

- Step 2: Activate Write Enable signal

The Von Neumann Model



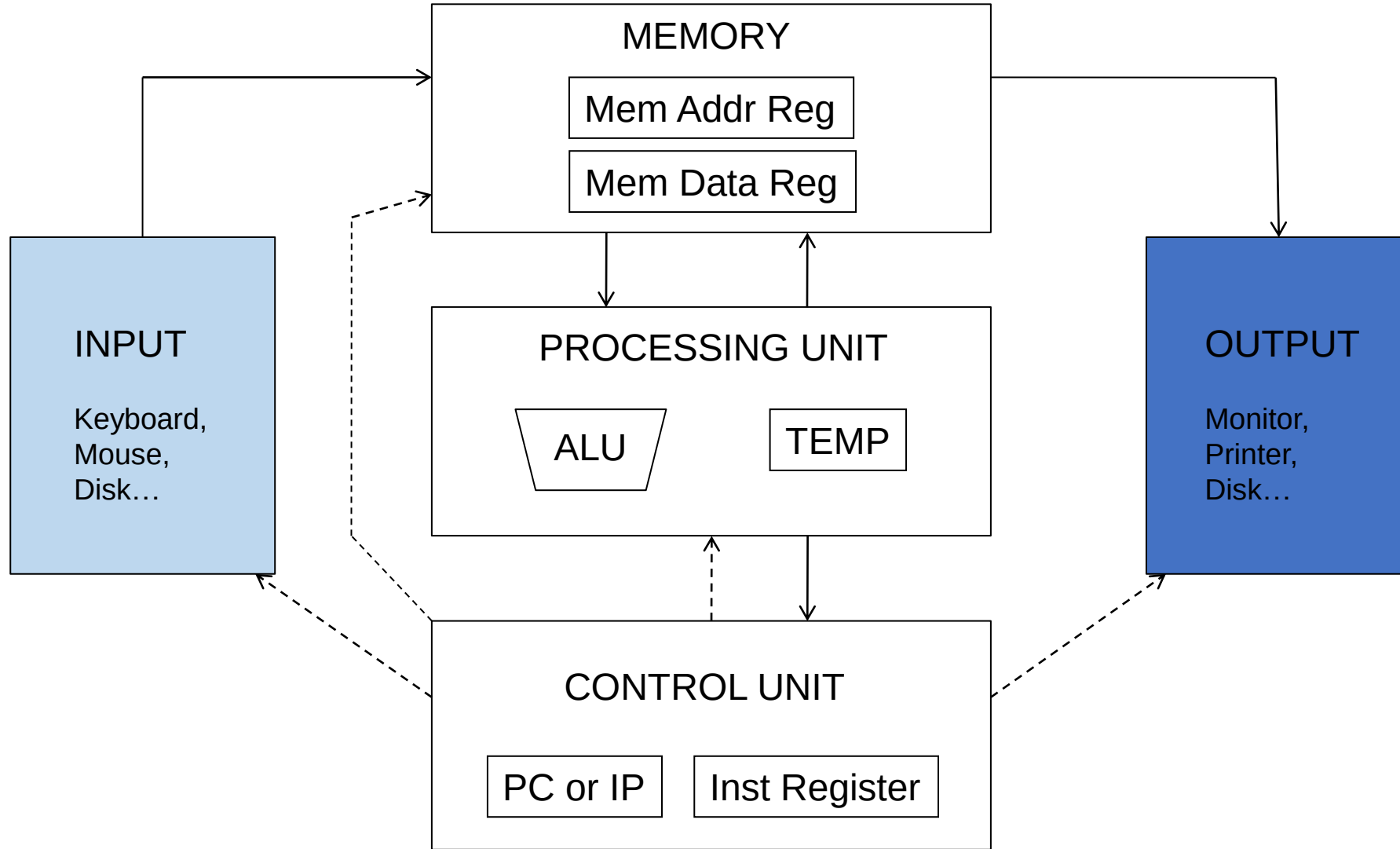
Processing Unit

- The processing unit can consist of many **functional units**
- We start with a simple **Arithmetic and Logic Unit (ALU)**, which executes computations
 - **LC-3**: ADD, AND, NOT (XOR in LC-3b)
 - **MIPS**: add, sub, mult, and, nor, sll, slr, slt...
- The ALU processes quantities that are referred to as **words**
 - **Word length** in LC-3 is 16 bits
 - In MIPS it is 32 bits
- Temporary storage: **Registers**
 - E.g., to calculate $(A+B)*C$, the intermediate result of $A+B$ is stored in a register

Registers

- Memory is big but slow
- Registers
 - Ensure fast access to operands
 - Typically one register contains one word
- Register set or file
 - LC-3 has 8 general purpose registers (GPR)
 - R0 to R7: 3-bit register number
 - Register size = Word length = 16 bits
 - MIPS has 32 registers
 - Register size = Word length = 32 bits

The Von Neumann Model



Input and Output

- Many devices can be used for input and output

- They are called peripherals

- Input

- Keyboard

- Mouse

- Scanner

- Disks

- Etc.

- Output

- Monitor

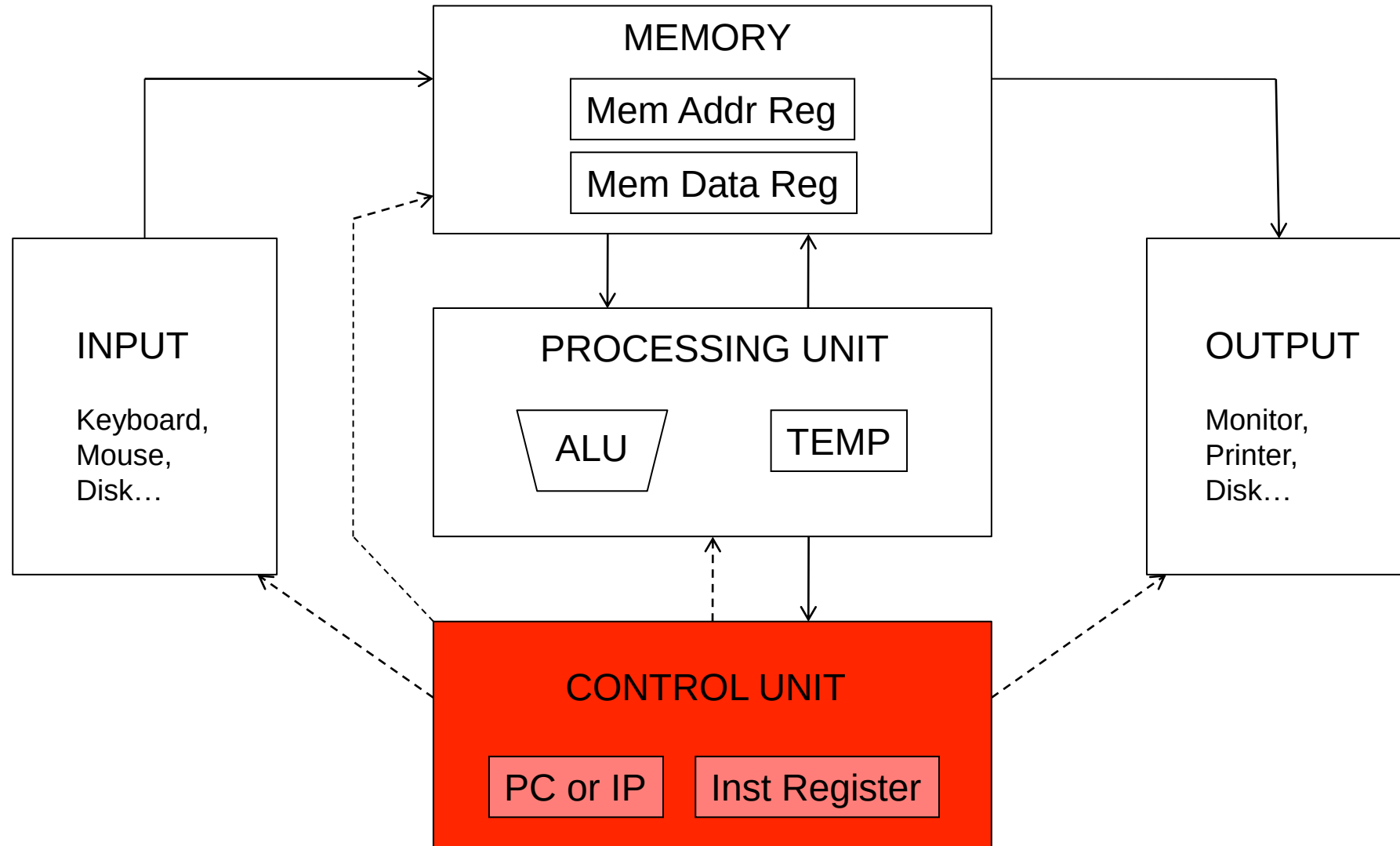
- Printer

- Disks

- Etc.

- In LC-3, we consider keyboard and monitor

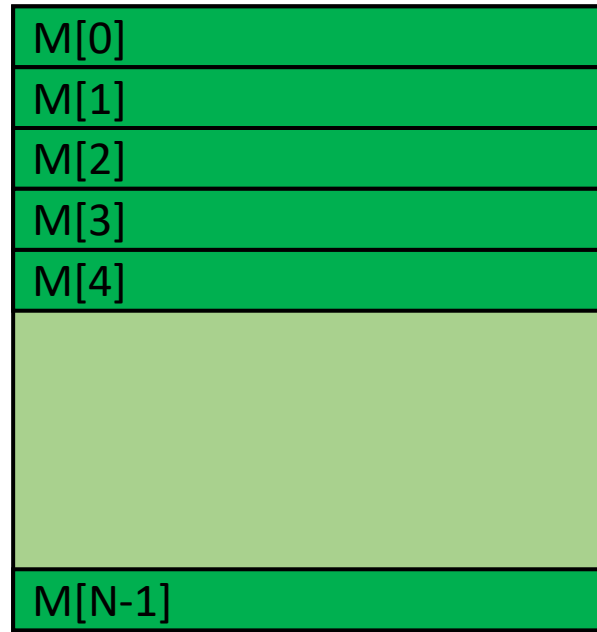
The Von Neumann Model



Control Unit

- The control unit is similar to the conductor of an orchestra
- It conducts the step-by-step process of executing (every instruction in) a program
- It keeps track of the instruction being executed with an instruction register (IR), which contains the instruction
- Another register contains the address of the next instruction to execute. It is called program counter (PC) or instruction pointer (IP)

Programmer Visible (Architectural) State



Memory
array of storage locations
indexed by an address



Registers

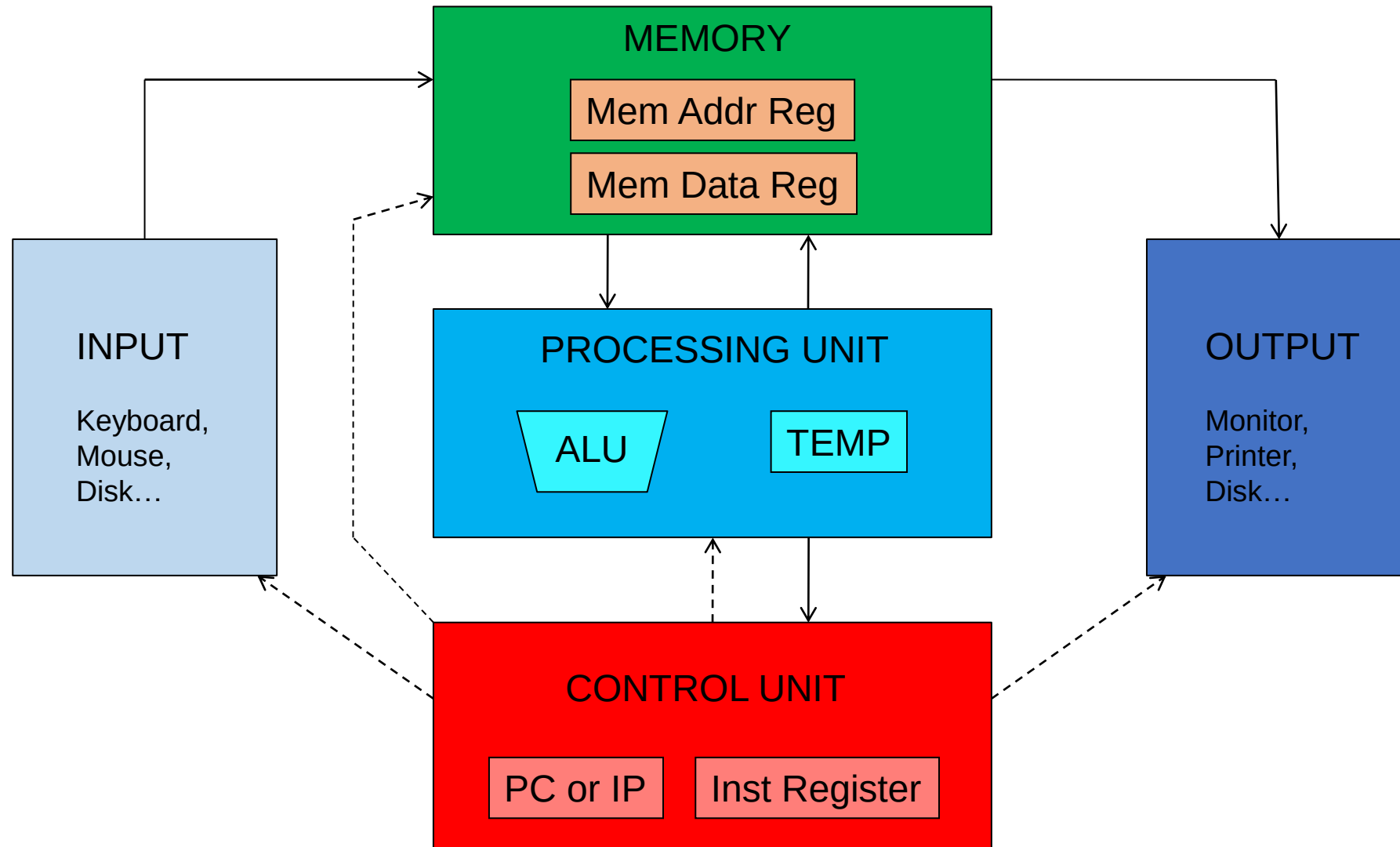
- given special names in the ISA (as opposed to addresses)
- general vs. special purpose

Program Counter

memory address
of the current instruction

Instructions (and programs) specify how to transform
the values of programmer visible state

The Von Neumann Model



Von Neumann Model: Two Key Properties

- Von Neumann model is also called *stored program computer* (instructions in memory). It has two key properties:
- **Stored program**
 - Instructions stored in a linear memory array
 - **Memory is unified** between instructions and data
 - The interpretation of a stored value depends on the control signals
- **Sequential instruction processing**
 - One instruction processed (fetched, executed, completed) at a time
 - **Program counter (instruction pointer)** identifies the current instruction
 - **Program counter is advanced sequentially** except for control transfer instructions

LC-3: A Von Neumann Machine

LC-3: A Von Neumann Machine

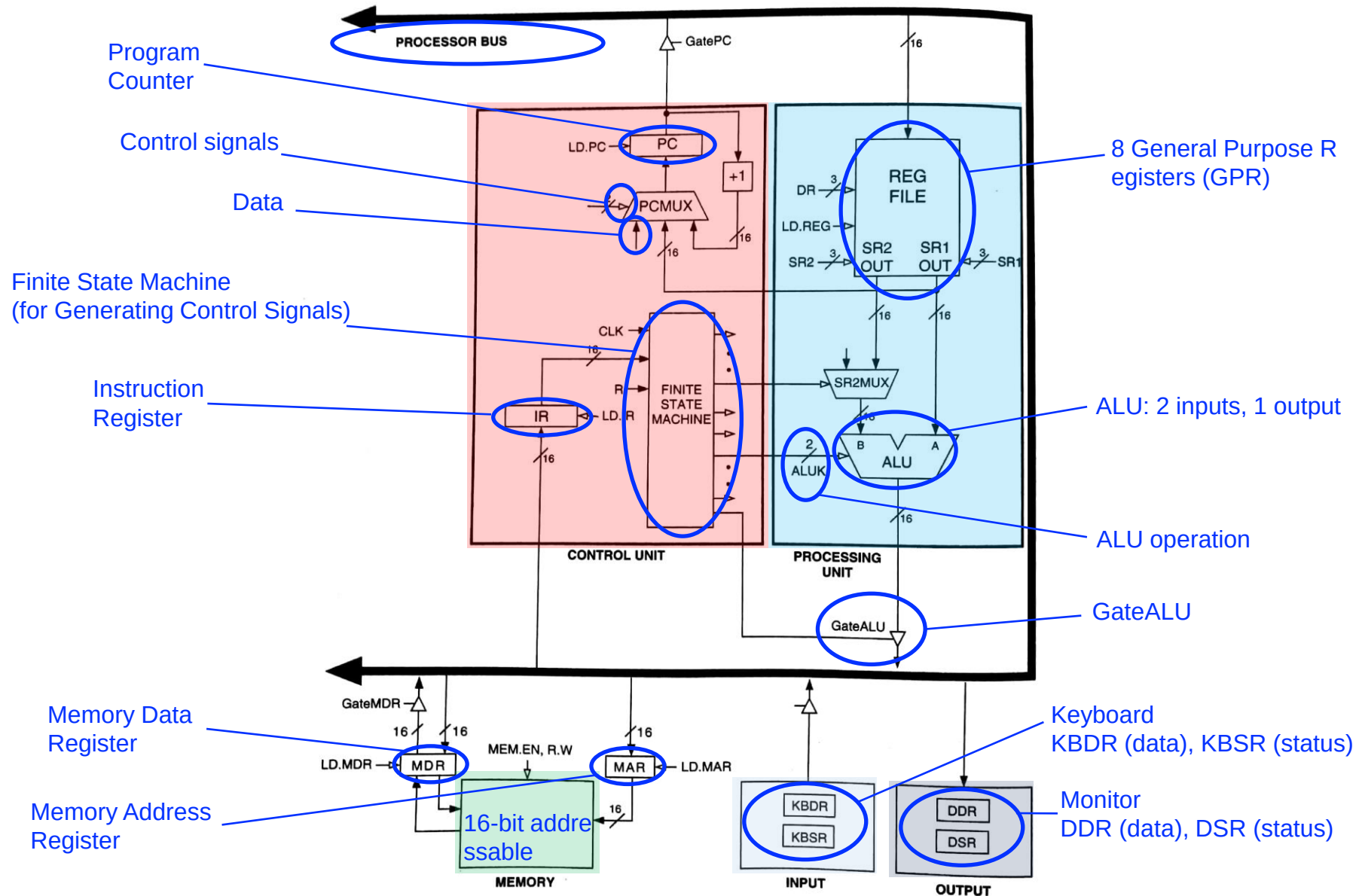


Figure 4.3 The LC-3 as an example of the von Neumann model

Stored Program & Sequential Execution

- Instructions and data are **stored in memory**
 - Typically **the instruction length is the word length**
- The processor fetches instructions from memory **sequentially**
 - Fetches one instruction
 - Decodes and executes the instruction
 - Continues with the next instruction
- The address of the current instruction is stored in the **program counter** (PC)
 - If **word-addressable** memory, the processor **increments the PC by 1** (in LC-3)
 - If **byte-addressable** memory, the processor **increments the PC by the word length** (4 in MIPS)
 - In MIPS the OS typically sets the PC to **0x00400000** (start of a program)

The Instruction

- An instruction the **most basic unit of computer processing**
 - **Instructions** are words in the language of a computer
 - **Instruction Set Architecture** (ISA) is the vocabulary
- The language of the computer can be written as
 - **Machine language**: Computer-readable representation (that is, 0's and 1's)
 - **Assembly language**: Human-readable representation
- We will look at **LC-3 instructions**
- Let us start with some example instructions

Instruction Types

- There are three main types of instructions
- Operate instructions
 - Execute instructions in the ALU
- Data movement instructions
 - Read from or write to memory
- Control flow instructions
 - Change the sequence of execution

An Example Operate Instruction

- Addition

High-level code

```
a = b + c;
```

Assembly

```
add a, b, c
```

- **add**: mnemonic to indicate the operation to perform
- **b, c**: **source operands**
- **a**: **destination operand**
- $a \leftarrow b + c$

Registers

- We map variables to registers

Assembly

```
add a, b, c
```

LC-3 registers

```
b = R1
```

```
c = R2
```

```
a = R0
```

From Assembly to Machine Code in LC-3

- Addition

LC-3 assembly

```
ADD R0, R1, R2
```

Field Values

OP	DR	SR1			SR2
1	0	1	0	00	2

Machine Code

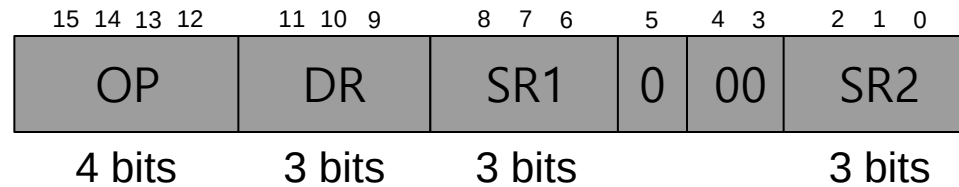
OP	DR	SR1			SR2
0 0 0 1	0 0 0	0 0 1	0	0 0	0 1 0
15 14 13 12	11 10 9	8 7 6	5	4 3	2 1 0

0x1042

Machine Code, in short (hexadecimal)

Instruction Format (or Encoding)

- LC-3



- OP = **opcode** (what the instruction does)
 - E.g., ADD = 0001
 - **Semantics:** $DR \leftarrow SR1 + SR2$
 - E.g., AND = 0101
 - **Semantics:** $DR \leftarrow SR1 \text{ AND } SR2$
- SR1, SR2 = source registers
- DR = destination register

Reading Operands from Memory

- With the **operate instructions**, such as addition, we tell the computer to **execute arithmetic (or logic) computations** in the ALU
- We also need instructions to **access the operands from memory**
- Next, we see how to **read (or load) from memory**
- **Writing (or storing)** is performed in a similar way, but we will talk about that later

Reading Word-Addressable Memory

- Load word

High-level code

```
a = A[i];
```

Assembly

```
load a, A, i
```

- **load**: mnemonic to indicate the load word operation
- **A**: base address
- **i**: offset
 - E.g., **immediate or literal** (a constant)
- **a**: destination operand
- **Semantics**: $a \leftarrow \text{Memory}[A + i]$

Load Word in LC-3

- LC-3 assembly

High-level code

```
a = A[2];
```

LC-3 assembly

```
LDR R3, R0, #2
```

$R3 \leftarrow \text{Memory}[R0 + 2]$

These instructions use a particular **addressing mode** (i.e., the way the address is calculated), called **base+offset**

Instruction Format With Immediate

- LC-3

LC-3 assembly

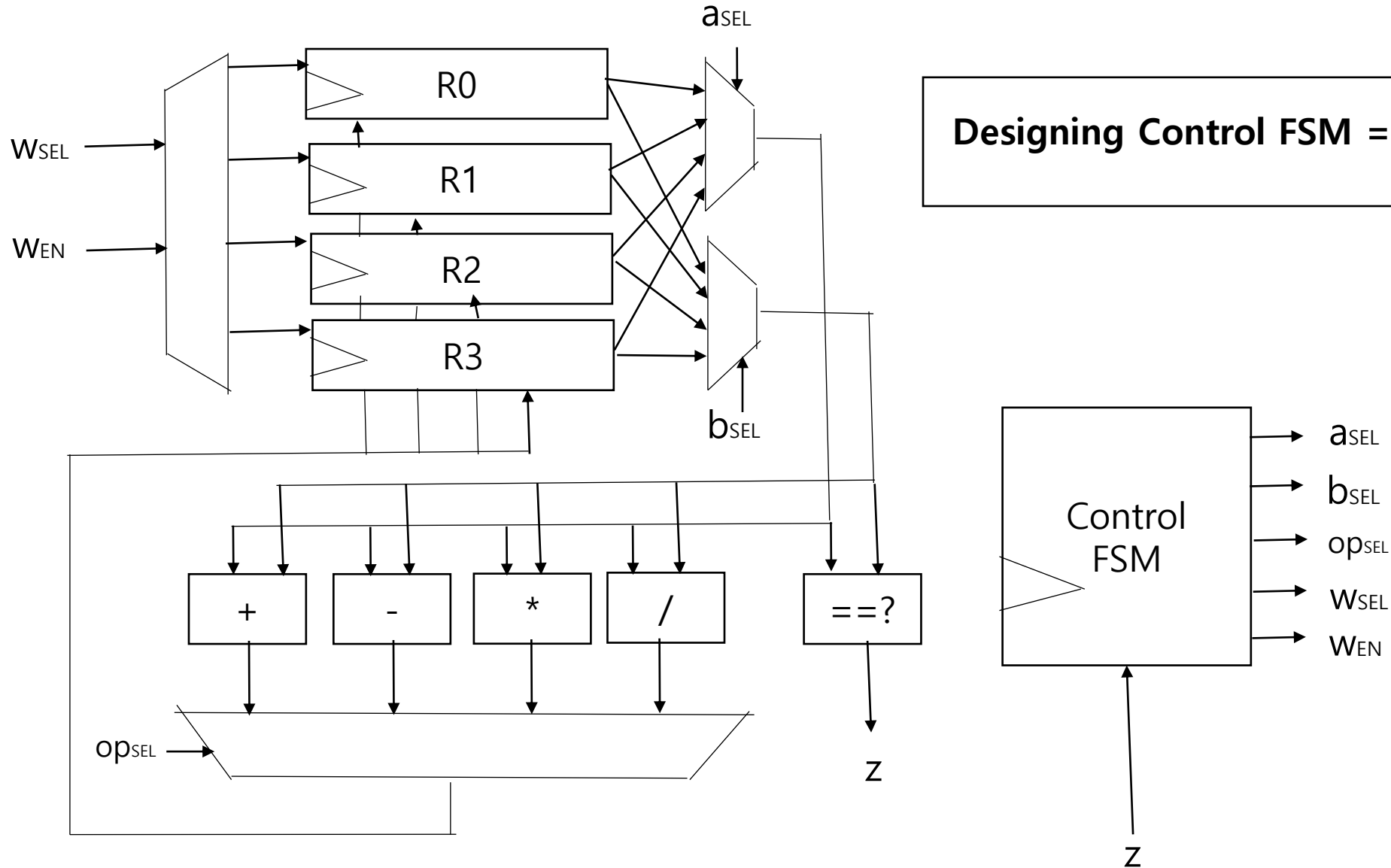
```
LDR R3, R0, #4
```

Field Values

OP		DR		BaseR		offset6	
6		3		0		4	
15	12	11	9	8	6	5	0

Von Neumann Model, ISA, and LC-3

Programmable Data Path



Readings

- *Introduction to Computing Systems: From Bits and Gates to C and Beyond* by Patt & Patel
 - Chapter 4, 5
 - Appendix A – LC3 ISA

How are These Instructions Executed?

- By using instructions we can speak the language of the computer
- Thus, we now know how to tell the computer to
 - Execute computations in the ALU by using add instruction
 - Access operands from memory by using the load word instruction
- But, how are these instructions executed on the computer?
 - The process of executing an instruction is called is the instruction cycle

The Instruction Cycle

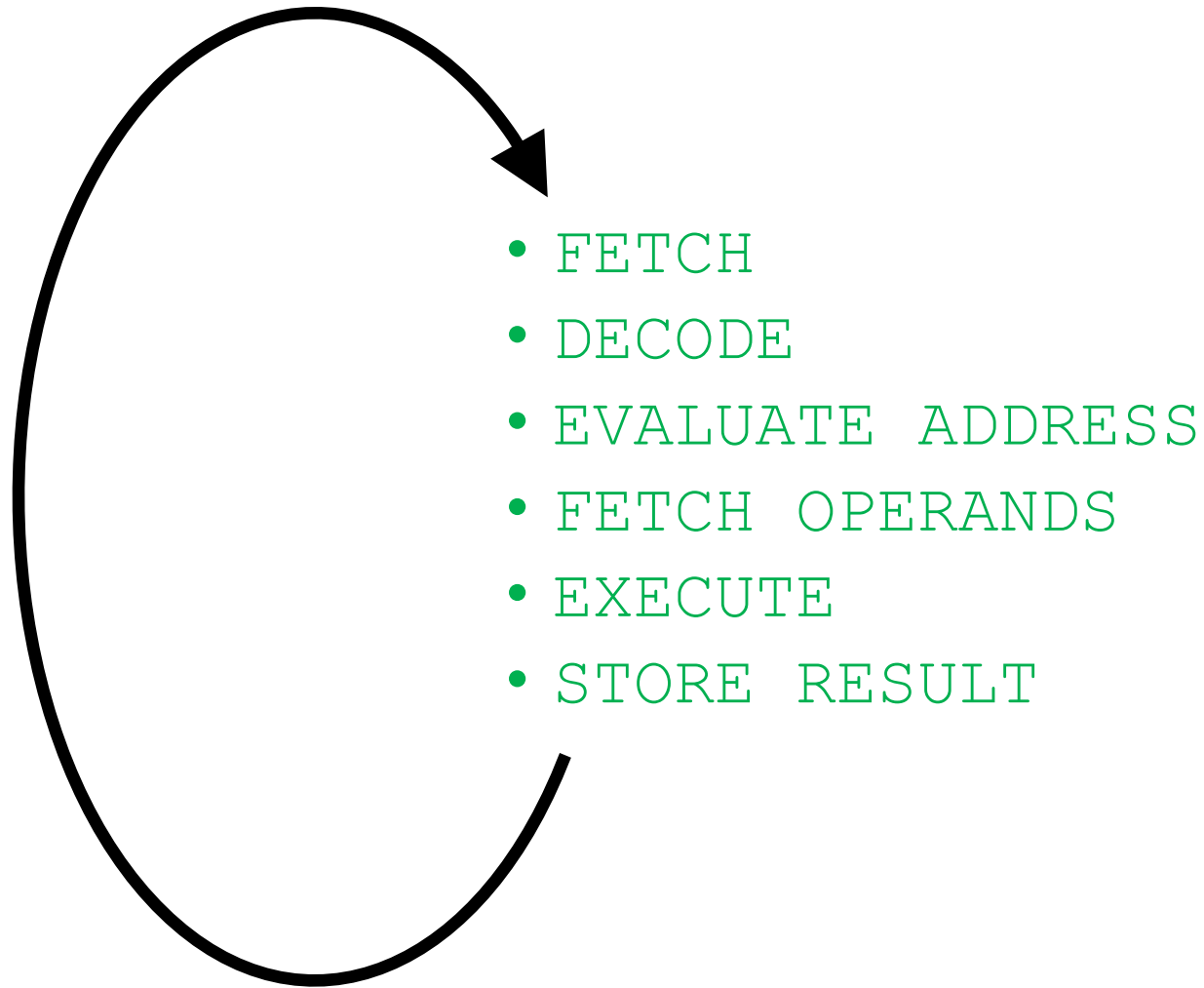
- The instruction cycle is a sequence of steps or **phases**, that an instruction goes through to be executed

- ☐ FETCH
- ☐ DECODE
- ☐ EVALUATE ADDRESS
- ☐ FETCH OPERANDS
- ☐ EXECUTE
- ☐ STORE RESULT

- Not all instructions have the six phases

- ☐ LDR does not require EXECUTE
- ☐ ADD does not require EVALUATE ADDRESS

After STORE RESULT, a New FETCH



FETCH

- The FETCH phase obtains the instruction from memory and loads it into the **instruction register**
- This phase is **common to every instruction type**
- **Complete description**
 - Step 1: **Load the MAR with** the contents of the **PC**, and simultaneously **increment the PC**
 - Step 2: Interrogate memory. This results the **instruction to be placed in the MDR**
 - Step 3: **Load the IR** with the contents of the **MDR**

FETCH in LC-3

Step 1: Load MAR and increment PC

Step 2: Access memory

Step 3: Load IR with the content of MDR

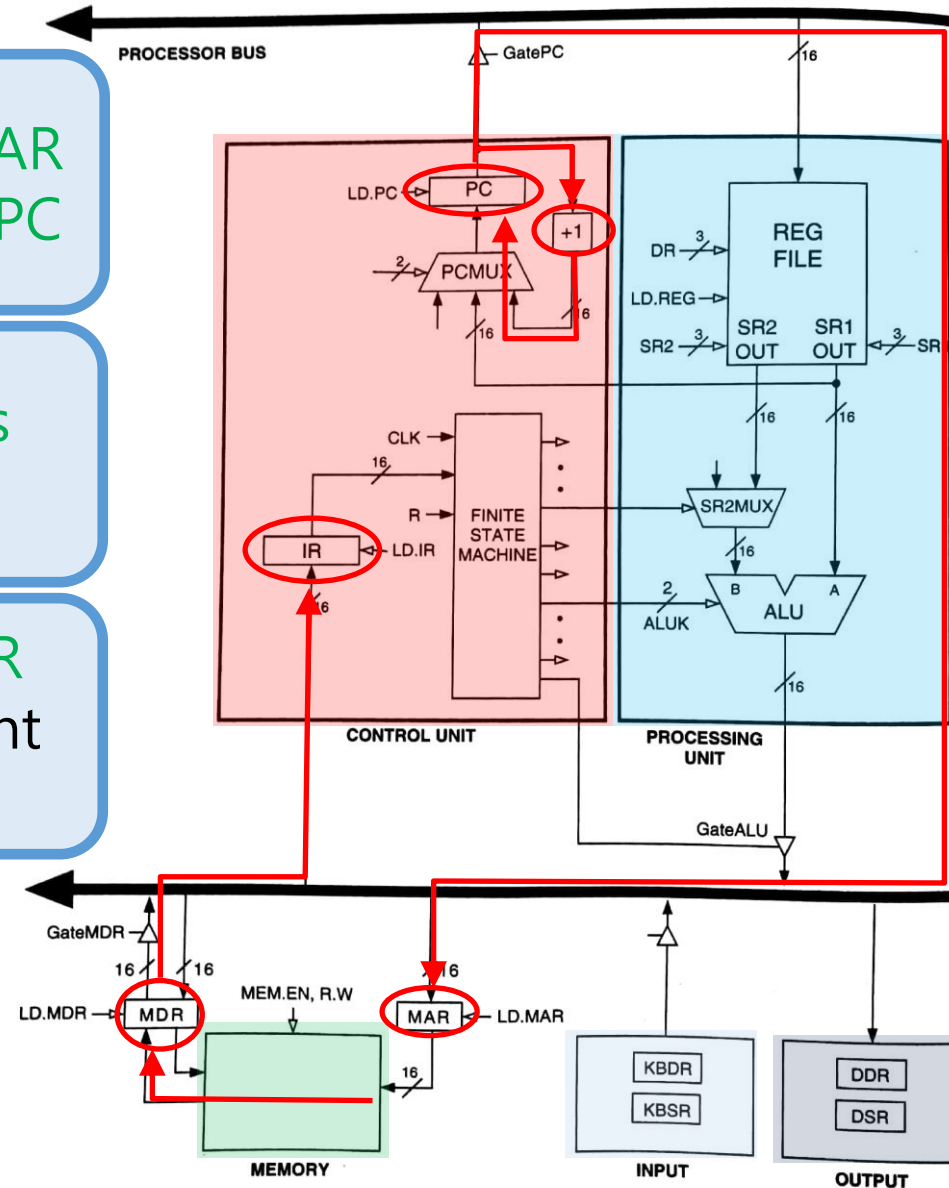


Figure 4.3 The LC-3 as an example of the von Neumann model

DECODE

- The DECODE phase identifies the instruction
- A decoder identifies which of the 16 opcodes is going to be processed
- The input is the four bits IR[15:12]
- The remaining 12 bits identify what else is needed to process the instruction

DECODE in LC-3

DECODE identifies
the instruction to
be processed

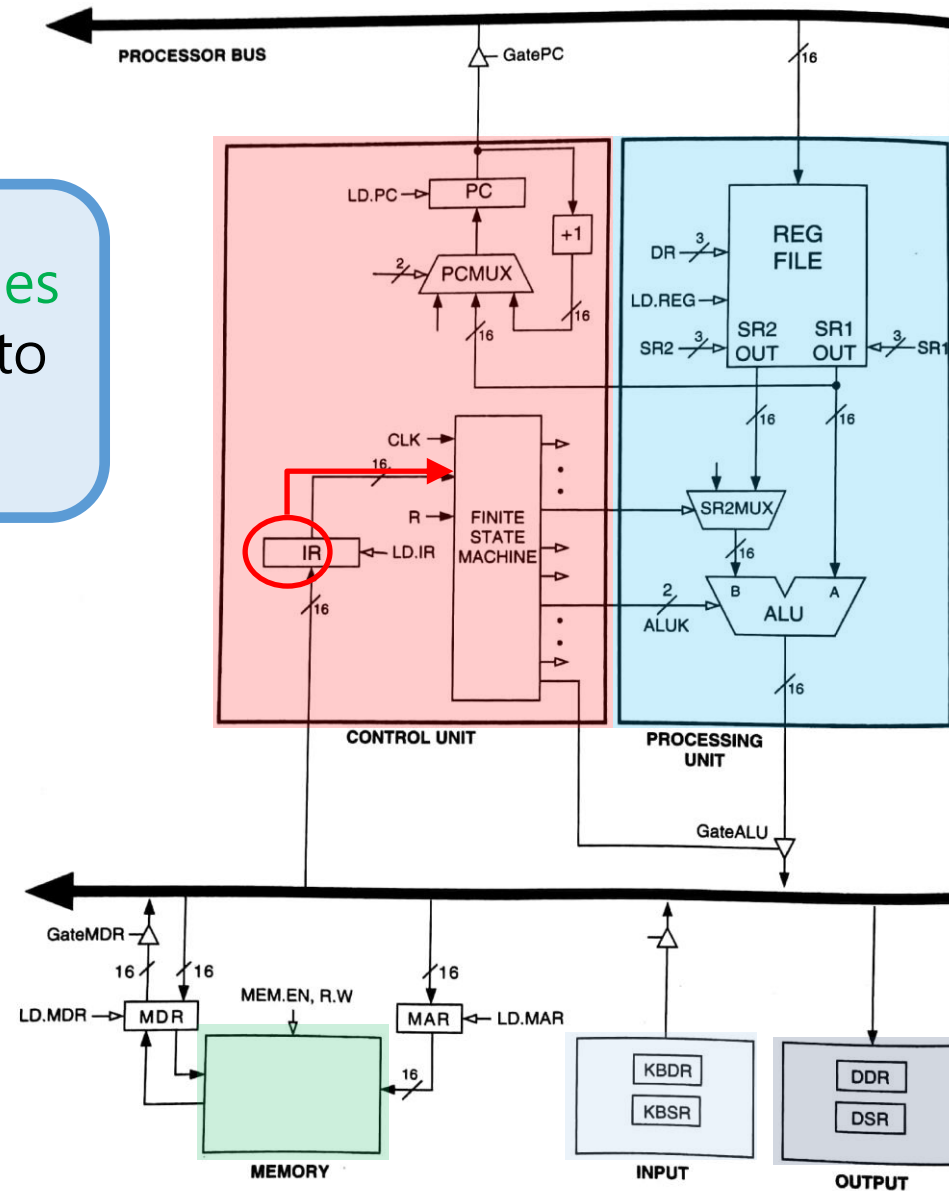


Figure 4.3 The LC-3 as an example of the von Neumann model

EVALUATE ADDRESS

- The EVALUATE ADDRESS phase computes the address of the memory location that is needed to process the instruction
- This phase is necessary in LDR
 - It computes the address of the data word that is to be read from memory
 - By adding an offset to the content of a register
- But not necessary in ADD

LDR calculates
the address by
adding a register
and an immediate

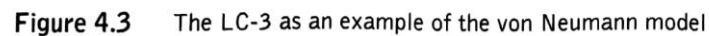


Figure 4.3 The LC-3 as an example of the von Neumann model

FETCH OPERANDS

- The FETCH OPERANDS phase obtains the source operands needed to process the instruction

- In LDR

- Step 1: Load MAR with the address calculated in EVALUATE ADDRESS

- Step 2: Read memory, placing source operand in MDR

- In ADD

- Obtain the source operands from the register file

FETCH OPERANDS in LC-3

LDR loads MAR(step 1),
and places the results
in MDR (step 2)

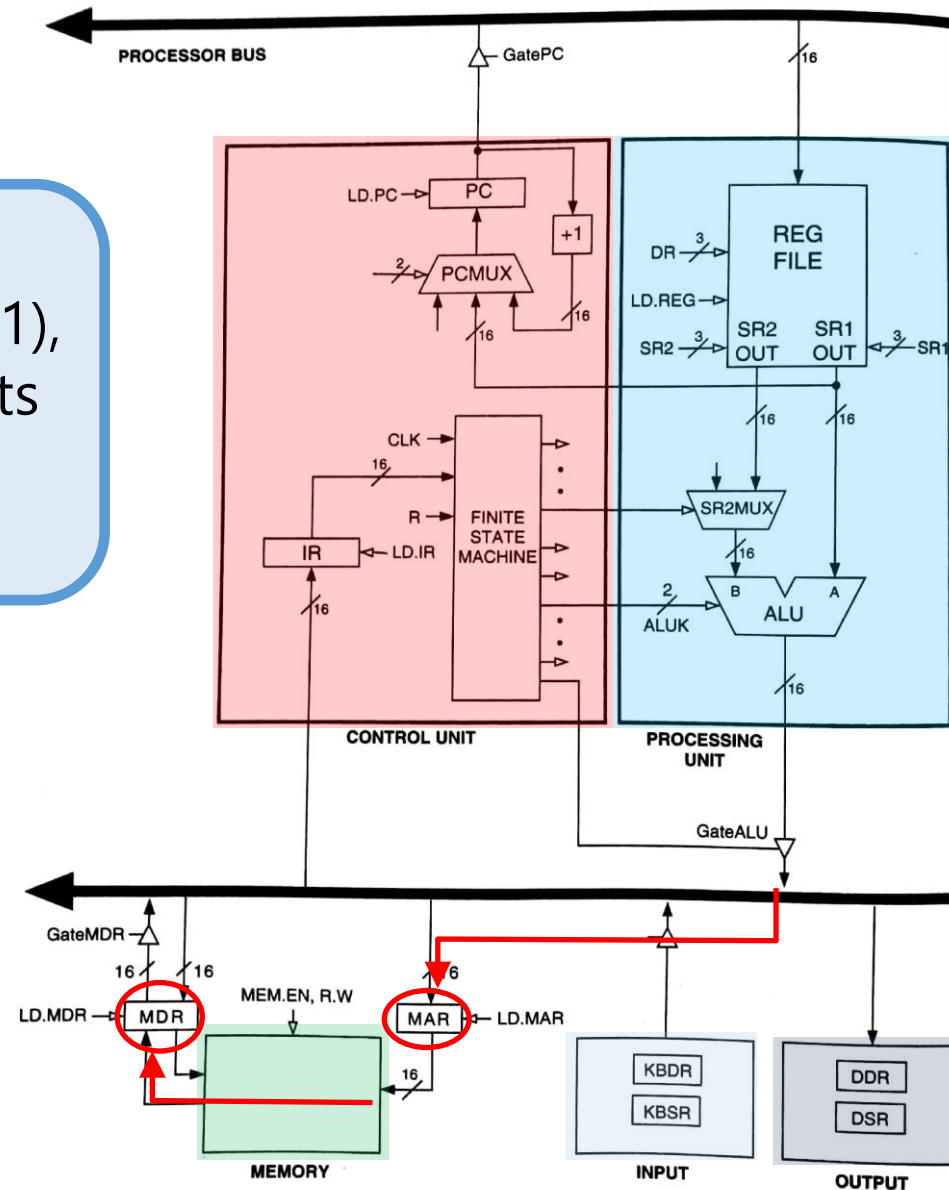


Figure 4.3 The LC-3 as an example of the von Neumann model

EXECUTE

- The EXECUTE phase **executes the instruction**
 - In ADD, it performs addition in the ALU

EXECUTE in LC-3

ADD adds SR1
and SR2

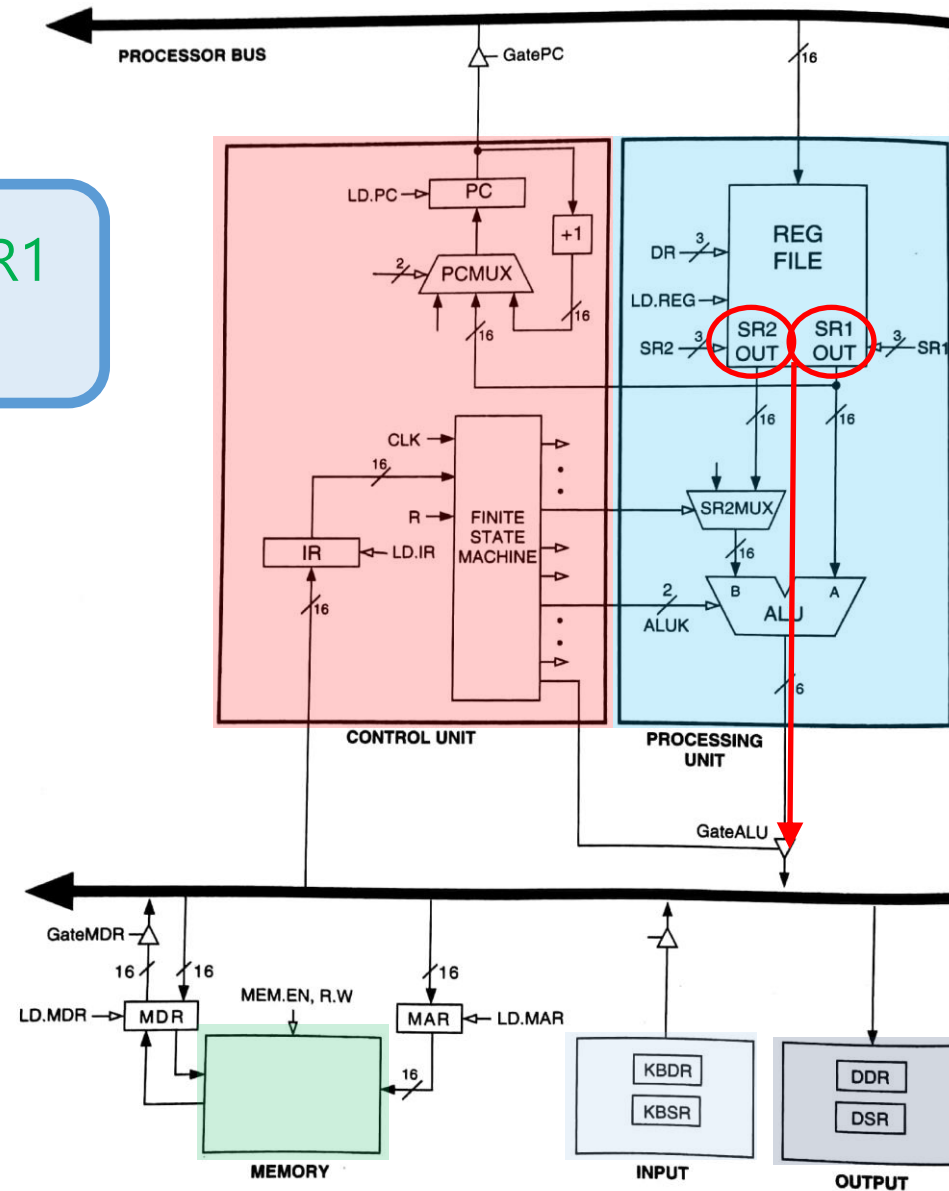


Figure 4.3 The LC-3 as an example of the von Neumann model

STORE RESULT

- The STORE RESULT phase writes to the designated destination
- Once STORE RESULT is completed, a new instruction cycle starts (with the FETCH phase)

LDR loads MDR
into DR

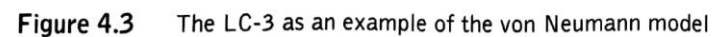
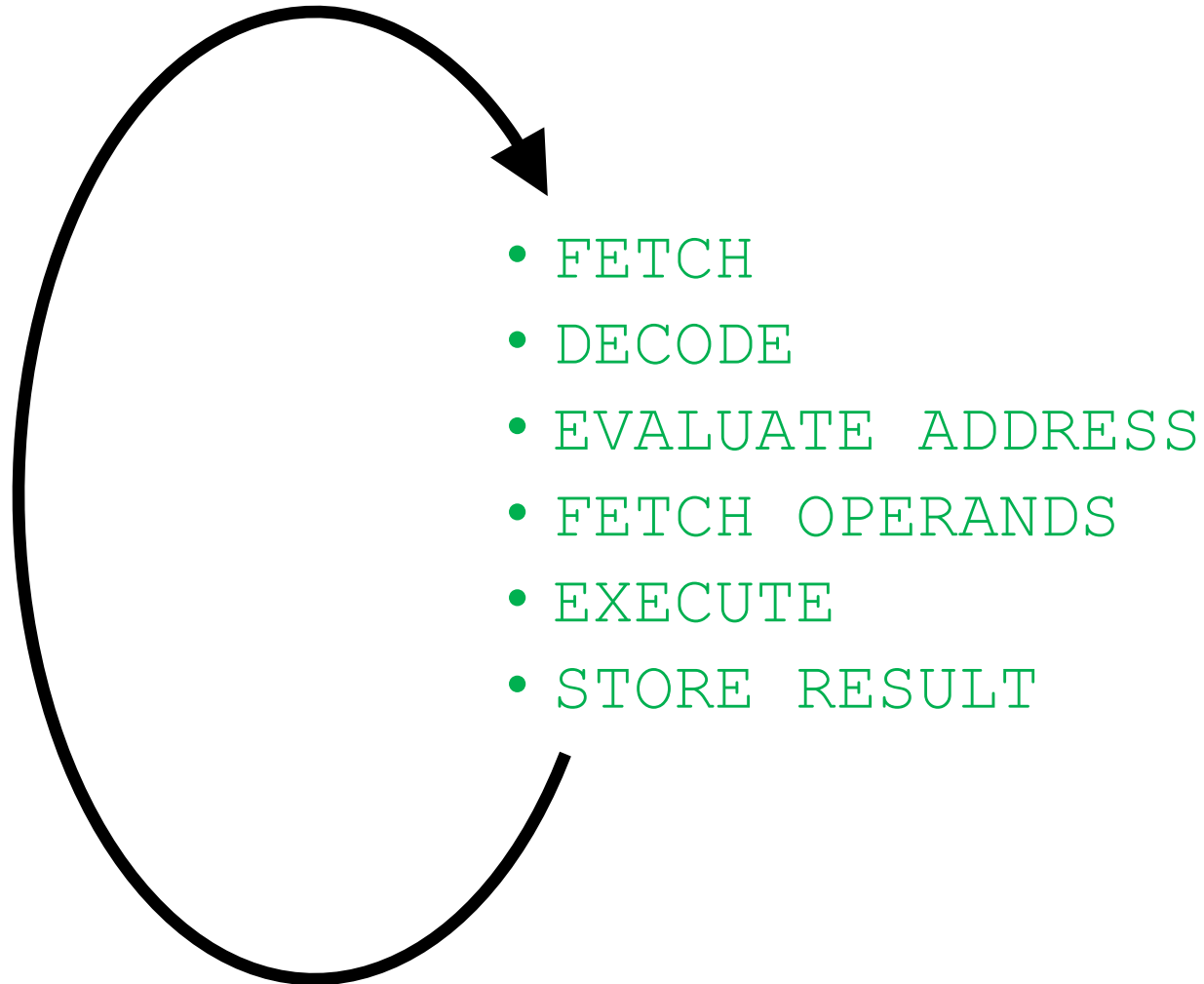


Figure 4.3 The LC-3 as an example of the von Neumann model

The Instruction Cycle



Changing the Sequence of Execution

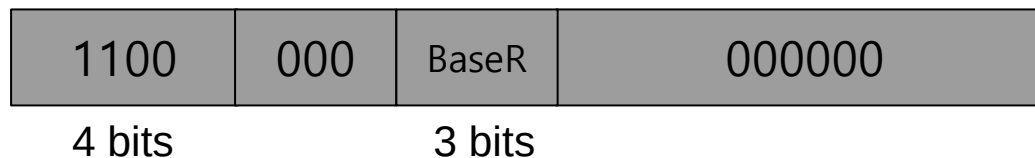
- A computer program **executes in sequence** (i.e., in program order)
 - First instruction, second instruction, third instruction and so on
- Unless we **change the sequence of execution**
- **Control instructions** allow a program to execute **out of sequence**
 - They can change the PC by loading it during the EXECUTE phase
 - That wipes out the incremented PC (loaded during the FETCH phase)

Jump in LC-3

- Unconditional branch or jump

- LC-3

JMP R2



- BaseR = Base register
- $PC \leftarrow R2$ (Register identified by BaseR)
- Variations
 - RET: special case of JMP where BaseR = R7
 - JSR, JSRR: jump to subroutine

This is **register**
addressing mode

LC-3 Data Path

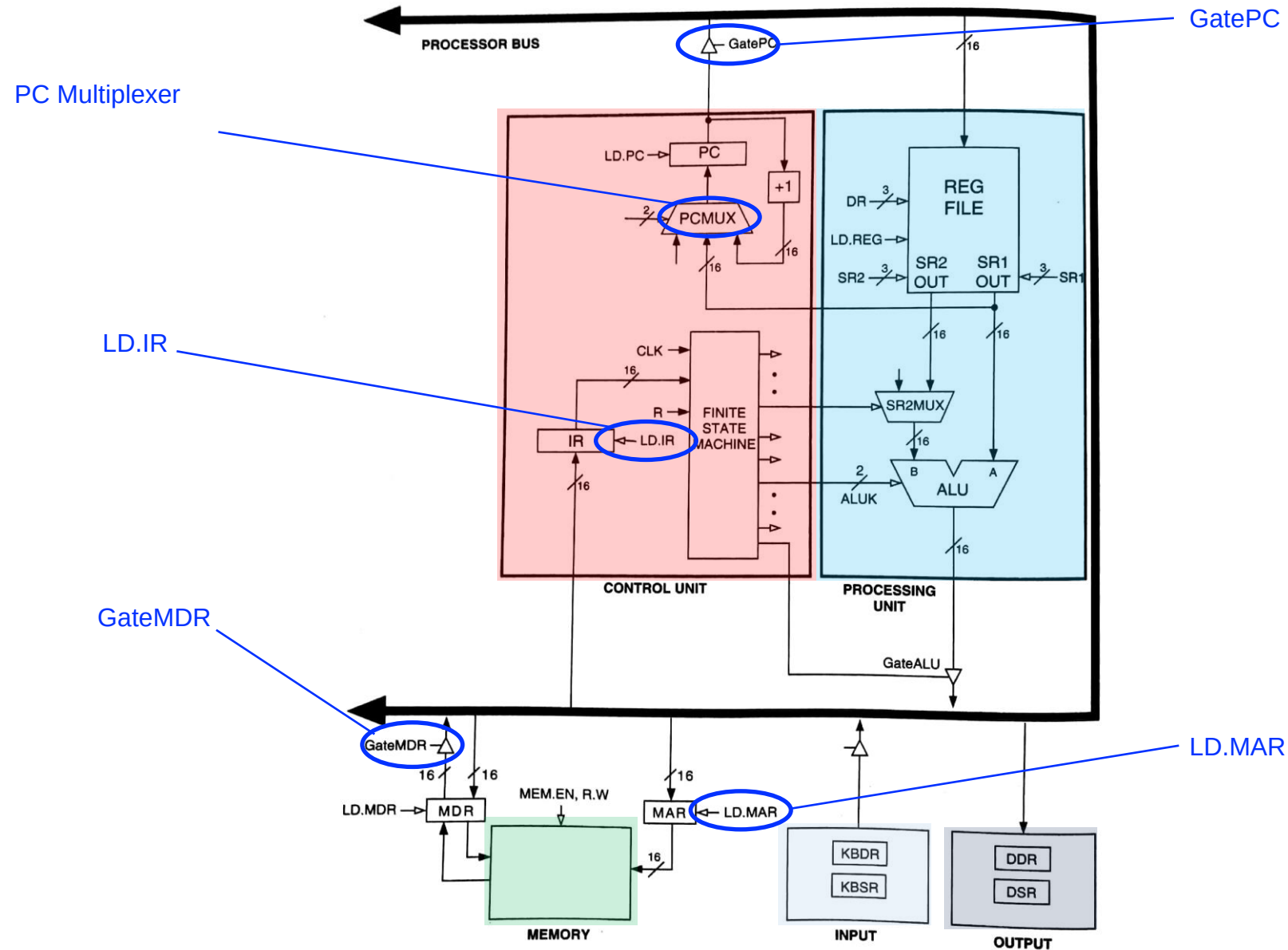


Figure 4.3 The LC-3 as an example of the von Neumann model